



CS4001NI Programming

50% Individual Coursework

2025-26 Autumn

Credit: 30

Student Name: Arjan Upadhayaya Timalisina

London Met ID: 25029344

College ID: NP01NT4AA250057

Assignment Due Date: March 9 2026

Assignment Submission Date: March 9 2026

Module Leader: Mr. Bikram Poudel

Word Count: 836

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1. Introduction
2. Class Diagram
3. Method Description
4. Testing
 - a. Prompt Execution
 - b. Prompt Rejection
 - c. Exhausting Monthly Quota for Personal Plan
 - d. Unlimited Prompts for Pro plan
5. Errors
 - a. Syntax Error
 - b. Runtime Error
 - c. Logical Error
6. Conclusion

1. Introduction

This coursework is about creating a real world solution, providing subscription services for AI models for personal and professional services. It consists of providing number of prompts for personal use and number of slots to add members for professional plan. The coursework has been written in java language.

2. Class Diagram

Almodel
-modelName: String -price: double -parameterCount: int -contextWindowSize: String
+Almodel(String,double,String,int) +getModelName(): String +getPrice(): double +getParameterCount(): String +getContextWindowSize(): String +displayModelInfo(): String

PersonalPlan
+remainingPromptsInMonth: int
+PersonalPlan(String,double,int,String,int) +getremainingPromptsInMonth(): int +purchasePrompts(int): String +userPrompts(String,String): String +displayModelInfo(): String

ProPlan
+slotsAvailable: int +remainingPromptsInMonth: int
+ProPlan(int, String,double,int,String) +addMember(String): String +removeMember(String): String +userPrompts(String,String): String +displayModelInfo(): String

3. Method Description

Name	Return Type	Parameters	Description
getModelName()	String	N/A	Getter method for modelName
getPrice()	String	N/A	Getter method for price
getParameterCount()	String	N/A	Getter method for parameterCount
getContextWindowSize()	String	N/A	Getter method for ContextWindowSize
displayModelInfo()	String	N/A	Getter method for modelInfo
getremainingPromptsInMonth()	int	N/A	Getter method for remainingPromptsInMonth
userPrompts()	String	String,String	Method to get the prompt from the user
userPurchasePrompts()	String	int	Method to purchase more prompts for personal user
addMember()	String	String	Method to add another user in pro plan
removeMember()	String	String	Method to remove a user

4. Testing

a. Add Plan

Name:

Chatgpt

Pricing:

100

Parameter Count:

20

Context Window:

4k

Prompt Quota:

500

Initial Team Members:

1

Prompt Text:

Hello

Response Length:

20

Name of Member:

Index number:

Add Personal Plan

Display All

Add Pro Plan

Clear

Give a Prompt

Add Team Member

Remove Team Member

Name:

Chatgpt

Pricing:

100

Parameter Count:

20

Context Window:

4k

Prompt Quota:

500

Initial Team Members:

Prompt Text:

Response Length:

Name of Member:

Index number:

Add Personal Plan

Display All

Add Pro Plan

Clear

Give a Prompt

Add Team Member

Remove Team Member

Name:

Pricing:

Parameter Count:

Context Window:

Prompt Quota:

Initial Team Members:

Prompt Text:

Response Length:

Name of Member:

Message

 Pro Plan added.

Name:

Pricing:

Parameter Count:

Context Window:

Prompt Quota:

Initial Team Members:

Prompt Text:

Response Length:

Name of Member:

Message

 Personal Plan added.

b. Prompt

Name:	
Pricing:	
Parameter Count:	
Context Window:	
Prompt Quota:	
Initial Team Members:	
Prompt Text:	Hello
Response Length:	
Name of Member:	
Index number:	1

Add Personal Plan	Display All	Add Pro Plan
Clear	Give a Prompt	Add Team Member
		Remove Team Member

c. Member for ProPlan only

Name of Member:	ram
Index number:	0

Add Personal Plan	Display All	Add Pro Plan
Clear	Give a Prompt	Add Team Member
		Remove Team Member

Index: 0
PersonalPlan@4a49e03a

Index: 1
ProPlan@6452fbbb

Error

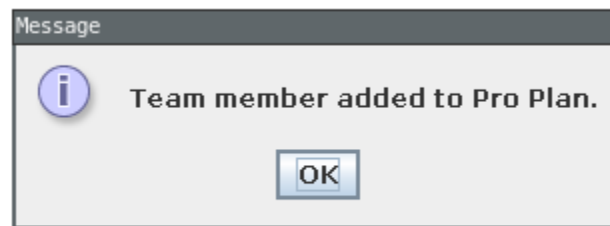
 Team collaboration is only available for Pro Plan subscriptions.

OK

Response Length:

Name of Member:

Index number:



d. Display all plans

Name:
 Pricing:
 Parameter Count:
 Context Window:
 Prompt Quota:
 Initial Team Members:
 Prompt Text:
 Response Length:
 Name of Member:
 Index number:

Index: 0
 PersonalPlan@20bd2d7e
 Index: 1
 ProPlan@3bc1d231

5. Conclusion

This project has helped me learn about the various use of java in daily use and how the concept of OOP can be used to solve any complex problem. Therefore I

learned about various errors and how to fix them as well. I also learned how to debug and find simple solutions for various logical problems.

6. References

- a. Java Documentation Oracle
<https://docs.oracle.com/en/java/>
- b. My second teacher
[My Second Teacher](#)

7. Appendix

a. AIModel

```
public class AIModel
{
    // instance variables - replace the example below with your own
    String modelName;
    double price;
    int parameterCount;
    String contextWindowSize;

    /**
     * Constructor for objects of class AIModel
     */
    public AIModel(String modelName, double price, int
parameterCount, String contextWindowSize)
    {
        // initialise instance variables
        this.modelName = modelName;
        this.price = price;
        this.parameterCount = parameterCount;
```

```

        this.contextWindowSize = contextWindowSize;
    }

    public String getModelName() {
        return modelName;
    }

    public double getPrice() {
        return price;
    }

    public int getParameterCount() {
        return parameterCount;
    }

    public String getContextWindowSize() {
        return contextWindowSize;
    }

    public String displayModelInfo(){
        return "Model Name: " + modelName + "\nPrice: $" + price +
        "\nParameter Count: " + parameterCount + "\nContext Window
        Size: " + contextWindowSize;

    }
}

```

b. Personal Plan

```

public class PersonalPlan extends AIModel
{
    // instance variables - replace the example below with your own
    public int remainingPromptsinMonth;

    /**
     * Constructor for objects of class PersonalPlan
     */
    public PersonalPlan(String modelName, double price, int
parameterCount, String contextWindowSize , int
remainingPromptsinMonth)
    {
        // initialise instance variables
        super(modelName, price, parameterCount, contextWindowSize);
        this.remainingPromptsinMonth = remainingPromptsinMonth;
    }

    public int getremainingPromptsinMonth() {
        return remainingPromptsinMonth;
    }

    public String purchasePrompts(int amount) {
        if (amount > 0) {
            remainingPromptsinMonth += amount;

            return "Successfully purchased " + amount + " prompts.
Remaining prompts in month: " + remainingPromptsinMonth;
        } else {
            return "Invalid amount. Please enter a positive number of
prompts to purchase.";
        }
    }
}

```

```

    }

    public String userPrompts(String promptText, String
outputLength){
        if (remainingPromptsinMonth > 0) {
            remainingPromptsinMonth -= 1;

            return "Here is your response for the prompt: " + promptText + "
with output length: " + outputLength;

        } else {
            return "No more prompts available. Please purchase more
prompts.";
        }
    }

    public String displayPlanInfo(){
        return super.displayModelInfo() + "\nRemaining Prompts in
Month: " + remainingPromptsinMonth;
    }
}

```

c. ProPlan

```

import java.util.ArrayList;

public class ProPlan extends AIModel {
    public int slotsAvailable;
    public int initialSlots;
    public int remainingPromptsinMonth;
    public ArrayList<String> members = new ArrayList<>(); // NEW

```

```
public ProPlan(int slotsAvailable, String modelName, double price, int
parameterCount, String contextWindowSize) {
    super(modelName, price, parameterCount, contextWindowSize);
    this.slotsAvailable = slotsAvailable;
    this.initialSlots = slotsAvailable;
}
```

```
public String addMember(String memberName) {
    if (slotsAvailable > 0) {
        slotsAvailable -= 1;
        members.add(memberName); // NEW
        return "Successfully booked slot for " + memberName + ". Remaining
slots: " + slotsAvailable;
    } else {
        return "No slots available. Please try again later.";
    }
}
```

```
public String removeMember(String memberName) {
    if (members.isEmpty()) {
        return "There are no team members to remove.";
    }
    if (!members.contains(memberName)) {
        return "Member \"" + memberName + "\" was not found.";
    }
    members.remove(memberName); // NEW
    slotsAvailable += 1;
    return "Successfully removed " + memberName + ". Remaining slots: "
+ slotsAvailable;
```

```
}
```

```
public String displayPlanInfo() {  
    return super.displayModelInfo() + "\nSlots Available: " + slotsAvailable;  
}
```

```
public String userprompts(String promptText, String outputLength) {  
    return "Here is your response for the prompt: " + promptText + " with  
output length: " + outputLength + ". You have unlimited prompts";  
}  
}
```

d. Subscription GUI

```
import java.util.ArrayList;  
import javax.swing.*.*;  
import java.awt.*.*;  
import java.awt.event.*;  
  
public class SubscriptionGUI {  
    ArrayList<AIModel> models = new ArrayList<>();  
  
    // Text fields for input  
    JTextField modelNameField = new JTextField(20);  
    JTextField priceField = new JTextField(20);  
    JTextField parameterCountField = new JTextField(20);  
    JTextField contextWindowSizeField = new JTextField(20);  
    JTextField remainingPromptsInMonthField = new JTextField(20);  
    JTextField slotsAvailableField = new JTextField(20);
```

```
TextField promptTextField = new TextField(20);
TextField outputLengthField = new TextField(20);
TextField memberNameField = new TextField(20);
TextField indexField = new TextField(20);
```

```
TextArea displayArea = new TextArea(15, 40);
```

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new
SubscriptionGUI().createAndShowGUI());
}
```

```
private void createAndShowGUI() {
    JFrame frame = new JFrame("Subscription Form");
    JPanel mainPanel = new JPanel();
    mainPanel.setLayout(new BorderLayout());
```

```
// Center panel for labels and text fields
JPanel fieldsPanel = new JPanel();
fieldsPanel.setLayout(new GridBagLayout());
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(5, 2, 5, 2);
gbc.anchor = GridBagConstraints.CENTER;
gbc.fill = GridBagConstraints.NONE;
gbc.gridx = 0;
gbc.gridy = 0;
```

```
int textFieldWidth = 12;
modelNameField = new TextField(textFieldWidth);
```



```
priceField = new JTextField(textFieldWidth);
parameterCountField = new JTextField(textFieldWidth);
contextWindowSizeField = new JTextField(textFieldWidth);
remainingPromptsinMonthField = new JTextField(textFieldWidth);
slotsAvailableField = new JTextField(textFieldWidth);
promptTextField = new JTextField(textFieldWidth);
outputLengthField = new JTextField(textFieldWidth);
memberNameField = new JTextField(textFieldWidth);
indexField = new JTextField(textFieldWidth);
```

```
String[] labels = {
    "Name:", "Pricing:", "Parameter Count:", "Context Window:",
    "Prompt Quota:", "Initial Team Members:", "Prompt Text:",
    "Response Length:", "Name of Member:", "Index number:"
};
```

```
JTextField[] fields = {
    modelNameField, priceField, parameterCountField,
    contextWindowSizeField,
    remainingPromptsinMonthField, slotsAvailableField,
    promptTextField,
    outputLengthField, memberNameField, indexField
};
```

```
for (int i = 0; i < labels.length; i++) {
    gbc.gridx = 0;
    gbc.weightx = 0;
    gbc.anchor = GridBagConstraints.EAST;
    fieldsPanel.add(new JLabel(labels[i], SwingConstants.RIGHT),
gbc);
```

```
gbc.gridx = 1;
gbc.weightx = 0;
gbc.anchor = GridBagConstraints.WEST;
fieldsPanel.add(Box.createHorizontalStrut(10), gbc); // ' ' space
```

```
gbc.gridx = 2;
gbc.weightx = 0;
gbc.anchor = GridBagConstraints.WEST;
fieldsPanel.add(fields[i], gbc);
```

```
gbc.gridy++;
}
```

```
// Buttons in two rows
```

```
JPanel buttonsPanel = new JPanel();
buttonsPanel.setLayout(new GridBagLayout());
GridBagConstraints bgbc = new GridBagConstraints();
bgbc.insets = new Insets(5, 10, 5, 10);
bgbc.fill = GridBagConstraints.HORIZONTAL;
bgbc.gridy = 0;
```

```
JButton addPersonalBtn = new JButton("Add Personal Plan");
JButton addProBtn = new JButton("Add Pro Plan");
JButton displayAllBtn = new JButton("Display All");
JButton clearBtn = new JButton("Clear");
JButton givePromptBtn = new JButton("Give a Prompt");
JButton addTeamMemberBtn = new JButton("Add Team
Member");
```

```
        JButton removeTeamMemberBtn = new JButton("Remove Team  
Member");
```

```
    // First row
```

```
    bgbc.gridx = 0; bgbc.gridwidth=1;  
    buttonsPanel.add(addPersonalBtn, bgbc);  
    bgbc.gridx = 1;  
    bgbc.gridwidth=2;  
    buttonsPanel.add(displayAllBtn, bgbc);  
    bgbc.gridx = 3; bgbc.gridwidth=1;  
    buttonsPanel.add(addProBtn, bgbc);
```

```
    // Second row
```

```
    bgbc.gridy = 1; bgbc.gridwidth=1;  
    bgbc.gridx = 0;  
    buttonsPanel.add(clearBtn, bgbc);  
    bgbc.gridx = 1;  
    buttonsPanel.add(givePromptBtn, bgbc);  
    bgbc.gridx = 2;  
    buttonsPanel.add(addTeamMemberBtn, bgbc);  
    bgbc.gridx = 3;  
    buttonsPanel.add(removeTeamMemberBtn, bgbc);
```

```
    fieldsPanel.setBorder(BorderFactory.createEmptyBorder(20, 40,  
10, 40));
```

```
    buttonsPanel.setBorder(BorderFactory.createEmptyBorder(10,  
40, 20, 40));
```

```
    displayArea.setEditable(false);
```

```
JScrollPane scrollPane = new JScrollPane(displayArea);
```

```
mainPanel.add(fieldsPanel, BorderLayout.NORTH);
```

```
mainPanel.add(buttonsPanel, BorderLayout.CENTER);
```

```
frame.add(mainPanel, BorderLayout.NORTH);
```

```
frame.add(scrollPane, BorderLayout.CENTER);
```

```
frame.pack();
```

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
frame.setLocationRelativeTo(null);
```

```
frame.setVisible(true);
```

```
addPersonalBtn.addActionListener(e -> addPersonalPlan());
```

```
addProBtn.addActionListener(e -> addProPlan());
```

```
displayAllBtn.addActionListener(e -> displayAll());
```

```
clearBtn.addActionListener(e -> clearFields());
```

```
givePromptBtn.addActionListener(e -> givePrompt());
```

```
addTeamMemberBtn.addActionListener(e ->  
addTeamMember());
```

```
removeTeamMemberBtn.addActionListener(e ->  
removeTeamMember());
```

```
}
```

```
private int getValidatedIndex() {
```

```
    int displayNumber = -1;
```

```
    try {
```

```
        int input = Integer.parseInt(indexField.getText().trim());
```

```
        if (input >= 0 && input < models.size()) {
```

```
            displayNumber = input;
```

```

    } else {
        JOptionPane.showMessageDialog(null, "Index out of range.
Please enter a value between 0 and " + (models.size() - 1) + ".",
"Invalid Index", JOptionPane.ERROR_MESSAGE);
    }
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(null, "Please enter a valid
integer for the index.", "Invalid Input",
JOptionPane.ERROR_MESSAGE);
}
return displayNumber;
}

private void addPersonalPlan() {
    try {
        String modelName = modelNameField.getText().trim();
        double price = Double.parseDouble(priceField.getText().trim());
        int parameterCount =
Integer.parseInt(parameterCountField.getText().trim());
        String contextWindowSize =
contextWindowSizeField.getText().trim();
        int remainingPromptsinMonth =
Integer.parseInt(remainingPromptsinMonthField.getText().trim());
        PersonalPlan plan = new PersonalPlan(modelName, price,
parameterCount, contextWindowSize, remainingPromptsinMonth );
        models.add(plan);
        JOptionPane.showMessageDialog(null, "Personal Plan
added.");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null, "Invalid input for
Personal Plan.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

```
}
```

```
private void addProPlan() {  
    try {  
        String modelName = modelNameField.getText().trim();  
        double price = Double.parseDouble(priceField.getText().trim());  
        int parameterCount =  
Integer.parseInt(parameterCountField.getText().trim());  
        String contextWindowSize =  
contextWindowSizeField.getText().trim();  
        int slotsAvailable =  
Integer.parseInt(slotsAvailableField.getText().trim());  
        ProPlan plan = new ProPlan(slotsAvailable,modelName, price,  
parameterCount, contextWindowSize);  
        models.add(plan);  
        JOptionPane.showMessageDialog(null, "Pro Plan added.");  
    } catch (Exception e) {  
        JOptionPane.showMessageDialog(null, "Invalid input for Pro  
Plan.", "Error", JOptionPane.ERROR_MESSAGE);  
    }  
}
```

```
private void displayAll() {  
    displayArea.setText("");  
    for (int i = 0; i < models.size(); i++) {  
        displayArea.append("Index: " + i + "\n" + models.get(i).toString()  
+ "\n\n");  
    }  
}
```

```

private void clearFields() {
    modelNameField.setText("");
    priceField.setText("");
    parameterCountField.setText("");
    contextWindowSizeField.setText("");
    remainingPromptsInMonthField.setText("");
    slotsAvailableField.setText("");
    promptTextField.setText("");
    outputLengthField.setText("");
    memberNameField.setText("");
    indexField.setText("");
}

```

```

private void givePrompt() {
    String prompt = promptTextField.getText().trim();
    String length = outputLengthField.getText().trim();
    boolean found = false;
    for (AIModel model : models) {
        if (model instanceof PersonalPlan) {
            ((PersonalPlan) model).userPrompts(prompt, length);
            JOptionPane.showMessageDialog(null, "API call made for
Personal Plan.");
            found = true;
            break;
        }
    }
    if (!found) {
        JOptionPane.showMessageDialog(null, "No Personal Plan
subscription found.", "Error", JOptionPane.ERROR_MESSAGE);
    }
}

```

```
}  
}
```

```
private void addTeamMember() {  
    int idx = getValidatedIndex();  
    if (idx != -1) {  
        AIModel model = models.get(idx);  
        if (model instanceof ProPlan) {  
            String memberName = memberNameField.getText().trim();  
            ((ProPlan) model).addMember(memberName);  
            JOptionPane.showMessageDialog(null, "Team member added  
to Pro Plan.");  
        } else {  
            JOptionPane.showMessageDialog(null, "Team collaboration is  
only available for Pro Plan subscriptions.", "Error",  
JOptionPane.ERROR_MESSAGE);  
        }  
    }  
}
```

```
private void removeTeamMember() {  
    int idx = getValidatedIndex();  
    if (idx == -1) return;  
  
    AIModel model = models.get(idx);  
  
    if (!(model instanceof ProPlan)) {  
        JOptionPane.showMessageDialog(null,  
            "Team collaboration is only available for Pro Plan  
subscriptions.",
```



```

        "Error", JOptionPane.ERROR_MESSAGE);
    return;
}

String memberName = memberNameField.getText().trim();

if (memberName.isEmpty()) {
    JOptionPane.showMessageDialog(null,
        "Please enter the name of the member to remove.",
        "Error", JOptionPane.ERROR_MESSAGE);
    return;
}

String result = ((ProPlan)
model).removeMember(memberName);
JOptionPane.showMessageDialog(null, result);
}}

```

8. GUI Pseudocode

```

function givePrompt:
    prompt = get text from prompt input field
    length = get text from output length input field
    found = false

```

for each model in models:

if model is a PersonalPlan:

call userPrompts(prompt, length) on model

show message "API call made for Personal Plan"

found = true

break

if found is false:

show error message "No Personal Plan subscription found"